POLITECHNIKA MORSKA SZCZECIN

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI

Architektura komputerów

Programowanie w asemblerze

Pętle programowe

mgr inż. Janusz Magaj

j.magaj@pm.szczecin.pl

Tematy

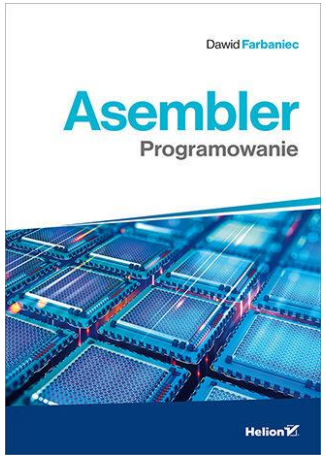
1. Wstęp do architektury komputerów
2. Architektura procesora
3. Reprezentacja i arytmetyka liczb całkowitych w procesorze
4. Reprezentacja i arytmetyka liczb zmiennoprzecinkowych w procesorze
5. **Programowanie w asemblerze**

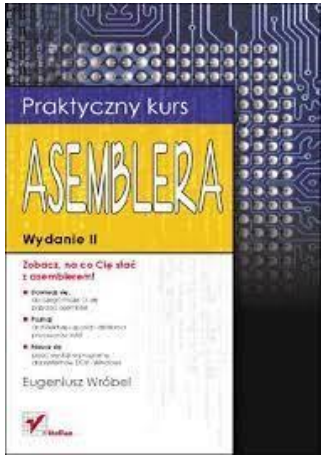
    1. Wstęp - asembler
    2. Operacje przypisania, operacje arytmetyczne
    3. Operacje warunkowe
    4. **Pętle programowe**
    5. Tablice, wskaźniki
    6. Stos, Funkcje, wywołanie funkcji
    7. Operacje zmiennoprzecinkowe

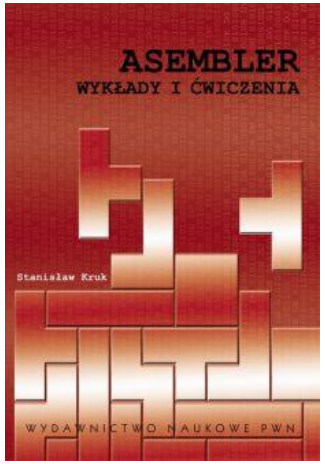
6. Budowa komputera PC

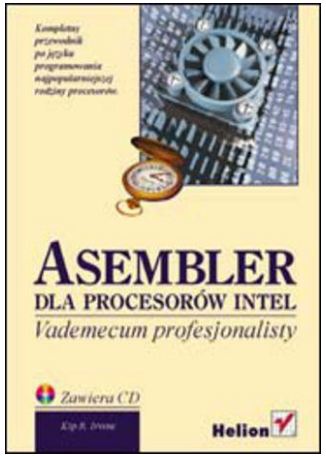
Literatura

- Farbaniec D., Asembler Programowanie, Helion, 2019
- Wróbel E., Praktyczny kurs ASEMBLERA, Helion, 2011
- Kruk S., Asembler Wykłady i ćwiczenia, WN PWN, 2009
- Pirogow V., Asembler. Podręcznik programisty, Helion, 2005
- Irvine K.R., Asembler dla procesorów Intel. Vademecum profesjonalisty, Helion, 2003
- Materiały firmowe – INTEL, AMD, Apple, Qualcomm









Pętle programowe

Pętle (ang. loop) to konstrukcje językowe służące do zdefiniowania szeregu instrukcji, które będą powtarzane wielokrotnie.

Podstawowe rodzaje pętli to:

- **for**,
- **while**
- **do .. while**.

Pętle programowe - FOR

W przypadku tej pętli z góry wiemy, ile razy instrukcje mają się wykonać.

Aby zdefiniować w pętlę for wykonującą się np. 10 razy zapiszemy:

```
for (i=1; i<=10; i++)
{
  // instrukcje, które będą się powtarzać
}
```

Zmienna **i** to liczba nazywana **zmienną sterująca** pętlą (albo **iteratorem pętli**).

W powyższym przykładzie w pierwszym wywołaniu pętli przypisujemy jej **wartość startową**: **i=1**.

W każdym przebiegu pętli **inkrementujemy** jej wartość (zapis: **i++**), czyli zwiększamy o dokładnie jeden.

Warunek **i<=10** określa **koniec iterowania** (powtarzania) instrukcji.

Pętla wykonuje się, dopóki warunek w środku zwraca wartość true (jest spełniony).
Jeśli choć raz ten warunek się nie spełni, pętla zostaje zakończona.

Pętle programowe - FOR

W przypadku tej pętli z góry wiemy, ile razy instrukcje mają się wykonać.

Pętla for może także zliczać w dół:

```
for (i=10; i>=0; i--)
{
  // instrukcje, które będą się powtarzać
}
```

Można także dodawać bądź odejmować w każdym wywołaniu pętli **dowolną wartość całkowitą**, zamiast **inkrementować** lub **dekrementować** (czyli **zmienić wartość o dokładnie jeden**):

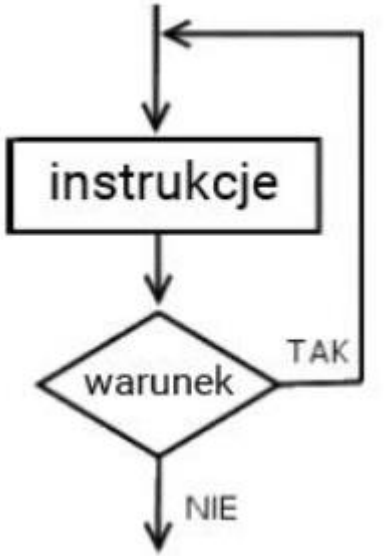
```
for (i=0; i<=10; i=i+2)
{
  alert(i);
}
```

Pętle programowe – WHILE, DO WHILE

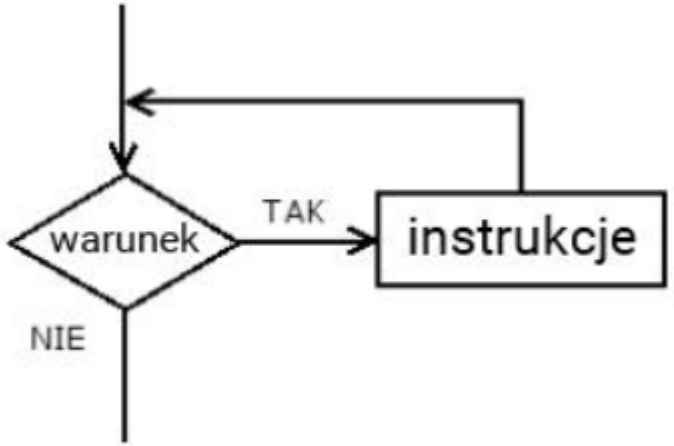
Pętle sterowane są warunkiem, a zatem instrukcje powtarzane są wielokrotnie dopóki warunek w nawiasie jest spełniony (prawdziwy).

Różnica między pętlą **while** i **do..while** polega na tym, iż w przypadku pętli **while** warunek sprawdzany jest na początku, zaś w **do..while** na końcu bloku instrukcji.

do..while



while



linie kodu zawarte w pętli **do..while** wykonają się zawsze przynajmniej jeden raz

Pętle programowe – WHILE, DO WHILE

Pętle sterowane są warunkiem, a zatem instrukcje powtarzane są wielokrotnie dopóki warunek w nawiasie jest spełniony (prawdziwy).

Zadanie:
Odgadujemy liczbę ("strzelamy" w skrypcie, jaka znajduje się w zmiennej liczba)

```
while (strzal != liczba)
{
  // instrukcje realizujące dalsze odgadywanie
  // liczby, gdy nie trafiono i liczby nie są równe!
}
```

Pętle programowe – asembler – pętla FOR

Instrukcja loop

- Instrukcja ułatwiająca tworzenie pętli podobnych do pętli for.
- Przyjmują one jako jedyny argument **etykietę**, do której należy wykonać skok.
- Wszystkie zmniejszają zawartość rejestru **ECX** o jeden.

LOOP etykieta *; wykonuje skok jeżeli ecx!=0*
LOOPZ etykieta *; wykonuje skok jeżeli ecx!=0 oraz ZF=1*
LOOPNZ etykieta *; wykonuje skok jeżeli ecx!=0 oraz ZF=0*

```
    mov eax, 0
    mov ecx, 10
poczatek_petli:
    add eax, ecx
    loop poczatek_petli
```

```
int suma=0;
for(int i=10; i!=0; --i)
  suma += i;
```

Pętle programowe – asembler – pętla FOR

Instrukcja skoku jmp

Schemat:

- Zapisanie w rejestrze wartości początkowych
- **Stworzenie etykiety określającej początek pętli**
- Sprawdzenie warunku wyjścia
- Skok warunkowy do etykiety wyjścia
- **Instrukcje wewnątrz ciała pętli**
- **Skok bezwarunkowy do początku pętli**

```
__asm poczatek:
__asm cmp ... ...;
__asm je / jge / jl / jle / jne ... koniec;

__asm jmp poczatek;
__asm koniec:
```

Pętle programowe – asembler – pętla FOR

Instrukcja skoku jmp

Debug | x86 | Lokalny debug

ConsoleApplication1.cpp

ConsoleApplication1 | (Globalny z

```cpp
#include <iostream>

int main()
{
    short i, j,k;
    i = 7; j = 9;
    __asm mov ax, 0;
    __asm mov cx, j;
    __asm start:
    __asm cmp cx, 0;
    __asm je koniec;
    __asm add ax, i;
    __asm sub cx, 1;
    __asm jmp start;
    __asm koniec:
    __asm mov k, ax;


    std::cout << k << " \n";
}
```

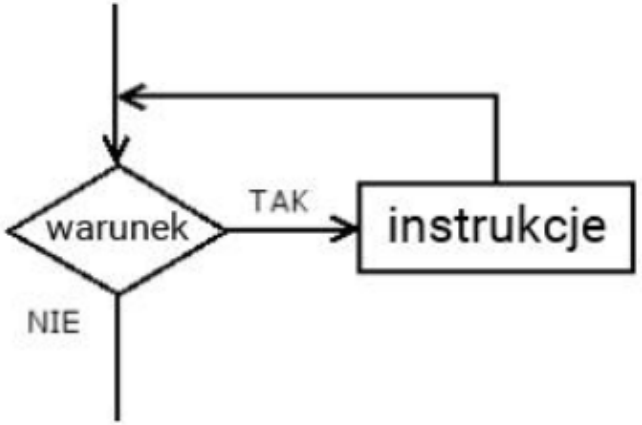
Konsola debugowania progra

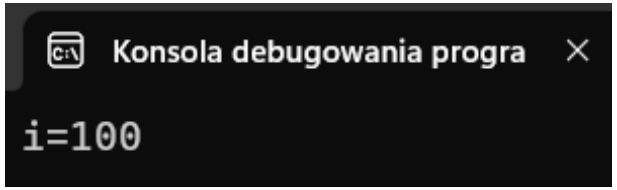
```
63

C:\vs2022_c\arch_komp\petla_for\pr1\ConsoleApplic
1620158712.
Aby automatycznie zamknąć konsolę po zatrzymaniu
znie zamknij konsolę po zatrzymaniu debugowania.
Naciśnij dowolny klawisz, aby zamknąć to okno...
```

Pętle programowe – asembler - WHILE

WHILE warunek;

```
int main()
{
    int i = 0;
    while (i < 100)
    {
        i++;
    }
    std::cout << "i=" << i << " \n";
}
```

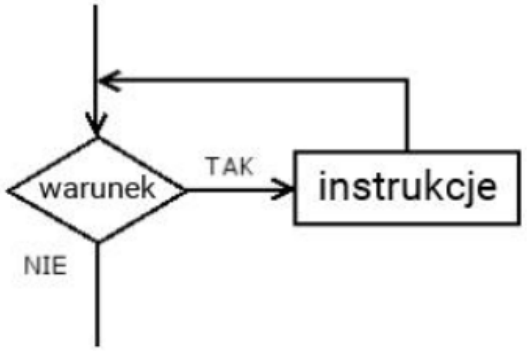




Pętle programowe – asembler - WHILE

WHILE warunek;

```
int main()
{
    int i = 0;
    while (i < 100)
    {
        i++;
    }
    std::cout << "i=" << i << " \n";
}
```



zad_petla1.asm*

```
1   .486
2   .model flat, stdcall
3   .stack 4096
4
5   ExitProcess PROTO, dwExitCode:DWORD
6
7   extern MessageBoxA@16 :proc
8
9   .data
10      i BYTE ?
11  .code
12  main proc
13          mov i, 0;
14   et1:   cmp i,100;
15          jge koniec_petli;
16          inc i;
17          jmp et1;
18  koniec_petli:
19  INVOKE ExitProcess, i
20  main    ENDP
21  END         main
```

Wyrażenie kontrolne 1

Wyszukiwanie (Ctrl+E)

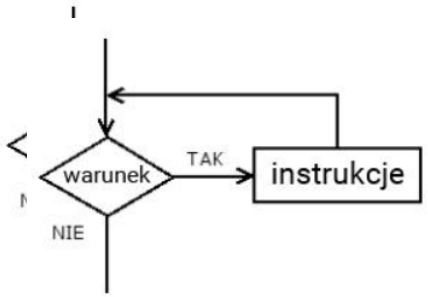
Nazwa	Wartość
i	100 'd'

Dane wyjściowe

Pokaż dane wyjściowe z: Debugowanie

```
Program „[592] Project1.exe” zakończył działanie z kodem 100 (0x64).
```

Pętle programowe – asembler - WHILE



Pętle można utworzyć wykorzystując instrukcje porównania i skoków

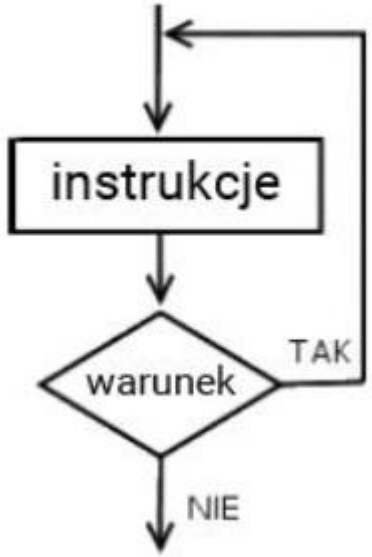
```
        optional initialization code
1:      IF not termination condition THEN BEGIN
                loop body
                GOTO 1;
        END;
```

Pętle programowe – asembler – DO WHILE

Pętla **do..while** sprawdza warunek zakończenia na końcu pętli, a nie na początku.

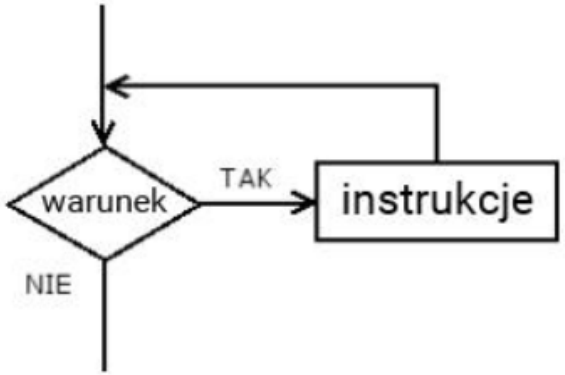
```
int main()
{
    int i = 0;
    do
        i++;
    while( i < 100);
    std::cout << "i=" << i << " \n";
}
```

```
Konsola debugowania progra
i=100
```



Pętle programowe – asembler – DO WHILE

```cpp
int main()
{
    int i = 0;
    do
        i++;
    while( i < 100);
    std::cout << "i=" << i << " \n";
}
```



zad_petla1.asm

```asm
.486
.model flat, stdcall
.stack 4096

ExitProcess PROTO, dwExitCode:DWORD

.data
    i BYTE ?
.code
main proc
        mov i, 0;
et1:    inc i;
        cmp i,100;
        jne et1;
        nop;

INVOKE ExitProcess, i
main    ENDP
END         main
```

Wyrażenie kontrolne 1

Wyszukiwanie (Ctrl+E)

Nazwa	Wartość
i	100 'd'

Dane wyjściowe

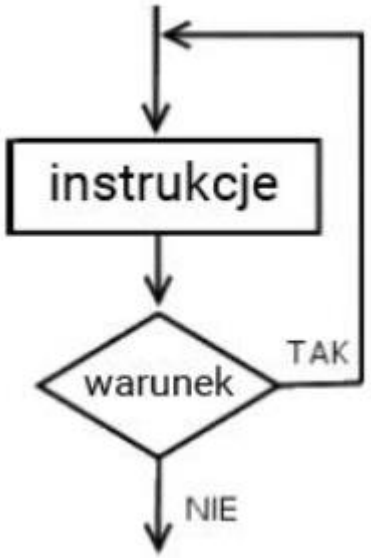
Pokaż dane wyjściowe z: Debugowanie

Program „[21048] Project1.exe” zakończył działanie z kodem 100 (0x64).

Pętle programowe – asembler – DO WHILE

Pętla **do..while** sprawdza warunek zakończenia na końcu pętli, a nie na początku.

```
optional initialization code
REPEAT
          loop body
UNTIL termination condition
```




```
        initialization code
1:                loop body
        IF NOT termination condition THEN GOTO 1
```

Pętle programowe – asembler – WHILE… DO

LOOP .. ENDLOOP

```
LOOP
        loop body
ENDLOOP;
```

- Pętle **while** i **do..while** testują warunek zakończenia na początku lub końcu pętli
- Może zaistnieć czasami potrzeba zakończenia w środku pętli.

```
1:
        READ(ch);
        IF ch = '.' THEN GOTO 2; (* Turbo Pascal supports BREAK! *)
        WRITE(ch);
        GOTO 1
2:
```

Asembler 86

```
LOOP1:  getc
        cmp     al, '.'
        je      EndLoop
        putc
        jmp     LOOP1
EndLoop:
```

Pętle programowe – asembler – FOR

for(;;)

```
For(int k=initial;k<final;k++)
For(int k=initial;k>final;k=k-2)
```

W pętli for instrukcje powtarzane są konkretną ilość razy.

Jest podana wartość inicjalizująca pętlę , wyrażenie sprawdzające wykonanie treści pętli oraz krok pętli (inkrementacja/dekrementacja)

Pętle programowe – asembler – FOR

```
                mov     var, start
FL:             mov     ax, var
                cmp     ax, stop
                jg      EndFor

; code corresponding to stmt goes here.

                inc     var
                jmp     FL
EndFor:
```

Pętle programowe – asembler – FOR

Przykład: Wyświetlenie **8** razy znaku ze zmiennej **ch**

```
FOR I := 0 to 7 do write(ch);
```

Asembler 86

```
                mov     cx, 7
LP:             mov     al, ch
                call    putc
                loop    LP
```

Wartość początkowa **0**

Pętle programowe – asembler – FOR

Wartość początkowa niezerowa

```
FOR var := start TO stop DO stmt;
```

Asembler 86

```
                mov     cx, stop
                sub     cx, start
                jl      SkipFor
                inc     cx
LP:             stmt
                loop    LP
SkipFor:
```

CX ← stop-start

A zmienna var ?

Pętle programowe – asembler – FOR

Pętla zostanie pominięta, jeśli stop jest mniejszy niż start.
W przeciwnym razie zostanie powtórzone (stop-start) + 1 razy

```
                mov     ax, start
                mov     var, ax
                mov     cx, stop
                sub     cx, ax
                jl      SkipFor
                inc     cx
LP:             stmt
                inc     var
                loop    LP
SkipFor:
```

var – zmienna z pętli

Pętle programowe – asembler – FOR

Wykorzystanie **rejestrów** w pętli **FOR**

Instrukcja skoku LOOP

pętla; zmniejsza wartość rejestru CX o jeden i jeśli CX jest większe od 0, to powoduje bezwarunkowy przeskok do etykiety

loop etykieta

```
                mov     cx, 8
Loop1:          mov     cx, 4
Loop2:          stmts
                loop    Loop2
                stmts
                loop    Loop1
```

Pętle programowe – asembler – FOR

Pierwsze rozwiązanie:

```
                mov     cx, 8
Loop1:          push    cx
                mov     cx, 4
Loop2:          stmts
                loop    Loop2
                pop     cx
                stmts
                loop    Loop1
```

Ze stosem **PUSH/POP**

Pętle programowe – asembler – FOR

Drugie rozwiązanie:

```
                mov     bx, 8
Loop1:          mov     cx, 4
Loop2:          stmts
                loop    Loop2
                stmts
                dec     bx
                jnz     Loop1
```

Z **Loop** oraz drugim rejestrem **BX**

Pętle programowe – asembler

Ulepszenia wydajności kodu

- mamy szybkie procesory
- rzadko spotkamy programy, który nie zawierają żadnych pętli.
- pętle są głównym źródłem problemów z wydajnością w programie i jednocześnie są one miejscem, w którym należy szukać rozwiązania przy próbie przyspieszenia oprogramowania.
- wszystkie poczynania mają na celu usunięcie zbędnych instrukcji z pętli, aby skrócić czas potrzebny do wykonania jednej iteracji pętli

Pętle programowe – asembler

Przyspieszenie zakończenia pętli

```
optional initialization code
REPEAT
          loop body
UNTIL termination condition
```

Repeat..until loop:

```
Initialization code
          Loop body
Test for termination
Code following the loop
```

Pętle programowe – asembler

Przyspieszenie zakończenia pętli

```
        optional initialization code
1:      IF not termination condition THEN BEGIN
                  loop body
                  GOTO 1;
        END;
```

```
Initialization code
Loop termination test
          Loop body
          Jump back to test
Code following the loop
```

Pętle programowe – asembler – FOR

Przyspieszenie zakończenia pętli

```
LOOP
        loop body
ENDLOOP;
```

Loop..endloop loop:

```
Initialization code
        Loop body, part one
        Loop termination test
        Loop body, part two
        Jump back to loop body part 1
Code following the loop
```

Pętle programowe – asembler

Przyspieszenie zakończenia pętli

Dwa rozwiązania tego samego problemu

```
SI := DI - 20;
while (SI <= DI) do
begin

        stmts
        SI := SI + 1;

end;
```

```
SI := DI - 20;
repeat

        stmts
        SI := SI + 1;

until SI > DI;
```

Pętle programowe – asembler

Przyspieszenie zakończenia pętli

```
SI := DI - 20;
while (SI <= DI) do
begin

        stmts
        SI := SI + 1;

end;
```

```
SI := DI - 20;
repeat

        stmts
        SI := SI + 1;

until SI > DI;
```

Rozwiązanie w asemblerze

```
        mov     si, di
        sub     si, 20
WL1:    cmp     si, di
        jnle    QWL
        stmts
        inc     si
        jmp     WL1
QWL:
```

```
        mov     si, di
        sub     si, 20
U:      stmts
        inc     si
        cmp     si, di
        jng     RU
```

Pętle programowe – asembler

Przyspieszenie zakończenia pętli

Rozwiązanie w asemblerze

```
        mov     si, di
        sub     si, 20
WL1:    cmp     si, di
        jnle    QWL
        stmts
        inc     si
        jmp     WL1
QWL:
```

```
        mov     si, di
        sub     si, 20
U:      stmts
        inc     si
        cmp     si, di
        jng     RU
```

Jak widać, testowanie warunku zakończenia na końcu pętli pozwoliło nam aby usunąć instrukcję **jmp** z pętli. Może to mieć znaczenie, jeśli ta pętla jest zagnieżdżona wewnątrz innych pętli.

Pętle programowe – asembler

Przyspieszenie zakończenia pętli

Rozwiązanie w asemblerze

```
        mov     si, di
        sub     si, 20
WL1:    cmp     si, di
        jnle    QWL
        stmts
        inc     si
        jmp     WL1
QWL:
```

```
        mov     si, di
        sub     si, 20
U:      stmts
        inc     si
        cmp     si, di
        jng     RU
```

Biorąc pod uwagę, że pętla będzie wykonano dokładnie 20 razy.

Zakładając, że **cx** jest dostępne, ta pętla łatwo redukuje się do:

```
        lea     si, -20[di]
        mov     cx, 20
WL1:    stmts
        inc     si
        loop    WL1
```

Pętle programowe – asembler

Przyspieszenie zakończenia pętli

```
WHILE (SI <= DI) DO BEGIN
          stmts
          SI := SI + 1;
END;
```

W tym konkretnym przykładzie nie mamy najmniejszego pojęcia, co si zawiera po wejściu w pętli. Dlatego nie możemy założyć, że ciało pętli wykona się przynajmniej raz.

Dlatego musimy wykonać test przed wykonaniem ciała pętli.
Próba może być umieszczony na końcu pętli z włączeniem pojedynczej instrukcji **jmp**:

```
                jmp     short Test
RU:             stmts
                inc     si
Test:           cmp     si, di
                jle     RU
```

Pętle programowe – asembler

Przyspieszenie zakończenia pętli

```
WHILE (SI <= DI) DO BEGIN
        stmts
        SI := SI + 1;
END;
```

```
        jmp     short Test
RU:     stmts
        inc     si
Test:   cmp     si, di
        jle     RU
```

```
        mov     si, di
        sub     si, 20
U:      stmts
        inc     si
        cmp     si, di
        jng     RU
```

- Chociaż kod jest tak długi jak oryginalna pętla **while**, instrukcja **jmp** jest wykonywana tylko raz, a nie przy każdym powtórzeniu pętli.
- Należy zauważyć, że ten niewielki wzrost wydajności jest uzyskany poprzez niewielką utratę czytelności.

Pętle programowe – asembler

Przyspieszenie zakończenia pętli

```
WHILE (SI <= DI) DO BEGIN
          stmts
          SI := SI + 1;
END;
```

```
                jmp     short Test
RU:             stmts
                inc     si
Test:           cmp     si, di
                jle     RU
```

- Czasami sekwencja kodu jest bliższa kodowi spaghetti niż oryginalna implementacja. Taka jest często cena niewielkiego wzrostu wydajności.
- Dlatego należy dokładnie przeanalizować swój kod, aby upewnić się, że plik z kodem zwiększającym wydajność jest wart utraty przejrzystości.
- Częściej bywa że, programiści poświęcają przejrzystość na rzecz wątpliwego wzrostu wydajności, tworząc niemożliwe do zrozumieć programy

Pętle programowe – asembler

Wykonywanie pętli wstecz

```
for I := 1 to 8 do
        K := K + I - J;
```

```
for I := 8 downto 1 do
        K := K + I - j;
```

Ze względu na charakter **flag** na 80x86, pętle, które wahają się od pewnej liczby do (lub do) zera są bardziej wydajne niż jakiekolwiek inne.

```
        mov     I, 1
FLP:    mov     ax, K
        add     ax, I
        sub     ax, J
        mov     K, ax
        inc     I
        cmp     I, 8
        jle     FLP
```

```
        mov     I, 8
FLP:    mov     ax, K
        add     ax, I
        sub     ax, J
        mov     K, ax
        dec     I
        jnz     FLP
```

dodatkowe porównanie

Niestety, nie można wymusić, aby wszystkie pętle działały wstecz.

Pętle programowe – asembler

Wykonywanie pętli wstecz

```
        mov     I, 8
FLP:    mov     ax, K
        add     ax, I
        sub     ax, J
        mov     K, ax
        dec     I
        jnz     FLP
```

Powyższy przykład zadziałał dobrze, ponieważ pętla przebiegała od ośmiu do jednego. Pętla zakończyła się, gdy zmienna kontrolna pętli osiągnęła zero.
Co się stanie, jeśli ty trzeba wykonać pętlę, gdy zmienna kontrolna pętli osiągnie zero?

Załóżmy na przykład, że powyższa pętla musiała zawierać się w przedziale od siedmiu do zera. Jeżeli wartość górna jest dodatnia, można zastąpić **instrukcję jnz** instrukcją **jns** powyżej, aby powtórzyć pętlę określoną liczbę razy

```
        mov     I, 7
FLP:    mov     ax, K
        add     ax, I
        sub     ax, J
        mov     K, ax
        dec     I
        jns     FLP
```

Kiedy zmienna **I** zmniejsza się od zera do minus jeden, ustawia flagę znaku **FS** i pętla się kończy.

Pętle programowe – asembler

Wykonywanie pętli wstecz

Pamiętaj, że niektóre wartości mogą wyglądać na dodatnie, ale są ujemne.

- Jeśli pętla zmienna sterująca jest bajtem, to wartości z zakresu 128..255 są ujemne.
- Podobnie 16-bitowe wartości z zakresu 32768..65535 są ujemne.
- Dlatego zainicjowanie zmiennej sterującej pętli dowolną wartością z zakresu 129..255 lub 32769..65535 (lub oczywiście zero) spowoduje, że pętla, która zakończy się po pojedynczym wykonaniu.

To może wpędzić cię w wiele kłopotów, jeśli jesteś nieostrożny

Pętle programowe – asembler – FOR

Obliczenia niezmienne w pętli

- Obliczenia niezmienne w pętli to obliczenia, które pojawiają się w pętli ale zawsze dają ten sam wynik.
- Nie musisz wykonywać takich obliczeń wewnątrz pętli.
- Możesz obliczyć je poza pętlą i odwołać się do wartości obliczeń wewnątrz

```
FOR I := 0 TO N DO
        K := K+(I+J-2);
```

```
temp := J-2;
FOR I := 0 TO N DO
        K := K+(I+temp);
```

Pętle programowe – asembler

Rozplątywanie pętli

W przypadku małych pętli, czyli takich, których ciało składa się tylko z kilku instrukcji, narzut wymagany do przetworzenia pętli może stanowić znaczny procent całkowitego przetwarzania czas.

```
FOR I := 3 DOWNTO 0 DO A [I] := 0;
```

```
        mov     I, 3
FLP:    mov     bx, I
        shl     bx, 1
        mov     A [bx], 0
        dec     I
        jns     FLP
```

Każde wykonanie pętli wymaga pięciu instrukcji.
Tylko jedna instrukcja wykonuje żądaną operację (przesunięcie zera do elementu A).
Pozostała czwórka instrukcji przekształcają zmienną kontrolną pętli w indeks w A i kontrolują powtarzanie pętli.
Dlatego potrzeba 20 instrukcji, aby wykonać operację wymaganą logicznie przez cztery.

Pętle programowe – asembler

Rozplątywanie pętli

```
FOR I := 3 DOWNTO 0 DO A [I] := 0;

                mov     I, 3
FLP:            mov     bx, I
                shl     bx, 1
                mov     A [bx], 0
                dec     I
                jns     FLP
```

Chociaż istnieje wiele ulepszeń, które moglibyśmy wprowadzić w tej pętli na podstawie przedstawionych do tej pory informacji, zastanówmy się dokładnie, co dokładnie robi ta pętla
— proste zapisanie czterech zer w A[0] do A[3].

Bardziej efektywnym podejściem jest użycie czterech **mov**, aby wykonać to samo zadanie.

```
mov     A, 0
mov     A+2, 0
mov     A+4, 0
mov     A+6, 0
```

Pętle programowe – asembler

Rozplątywanie pętli

```
FOR I := 3 DOWNTO 0 DO A [I] := 0;
```

Bardziej efektywnym podejściem jest użycie czterech **mov**, aby wykonać to samo zadanie.

```
mov     A, 0
mov     A+2, 0
mov     A+4, 0
mov     A+6, 0
```

Można poprawić szybkość wykonywania i rozmiar tego kodu, używając rejestru **ax** do przechowywania zera

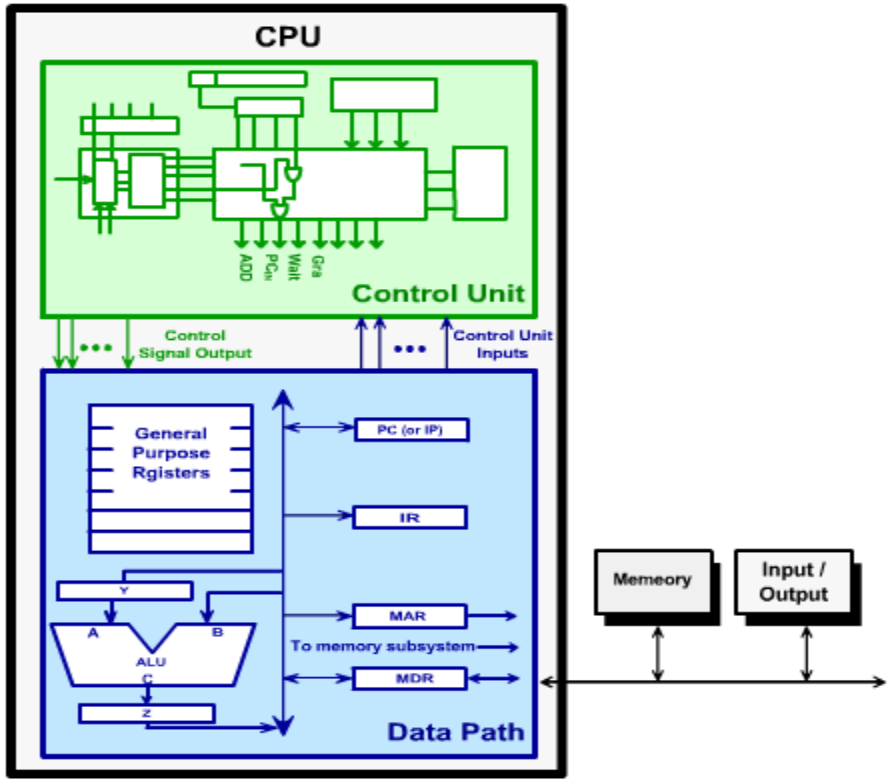
```
xor     ax, ax
mov     A, ax
mov     A+2, ax
mov     A+4, ax
mov     A+6, ax
```

Pętle programowe – asembler

Rozplątywanie pętli

```
xor     ax, ax
mov     A, ax
mov     A+2, ax
mov     A+4, ax
mov     A+6, ax
```

- Rozplątywanie pętli zapewnia imponującą poprawę wydajności, gdy liczba instrukcji wymaganych do sterowania pętlą (i obsługi innych operacje narzutowe) stanowią znaczny procent całkowitej liczby instrukcji w pętli.
- Gdyby powyższa pętla zawierała 36 instrukcji w ciele pętli (z wyłączeniem czterech instrukcji ogólnych), wówczas poprawa wydajności byłaby na poziomie lepszym tylko o 10% (w porównaniu z 300-400%, którymi cieszymy się teraz).
- Koszty rozwikłania pętli, tj. całego dodatkowego kodu, który należy wstawić do programu, szybko zwracają się, pomimo malejących zwrotów wydajności w miarę powiększania się ciała pętli lub zwiększania liczby iteracji.
- Wprowadzenie tego kodu do programu może dać najlepszy efekt.
- Dlatego rozplątywanie pętli jest techniką najczęściej stosowaną do małych pętli.



Dziękuję za uwagę